Universidad Icesi

Pruebas E2E, Despliegue y GraphQL

Computación en Internet 3 · Semana 15

Universidad Icesi

Universidad ICESI

1. Pruebas E2E

Comprobar que todo funciona junto

¿Qué es una prueba end-to-end?

QUÉ HACE

Abre un navegador real.

- Usa tu aplicación como lo haría una persona: escribe, hace clic y navega.
- No le importa cómo está escrito el código.

QUÉ COMPRUEBA

Que el sistema completo funciona junto.

- La interfaz, el servidor, las cookies y los datos.
- Lo que una prueba de una sola función no puede ver.

Universidad ICESI

Playwright, la herramienta del curso

Controla un navegador de verdad
Chromium, Firefox y WebKit, desde un script.

Espera por ti
No hay que adivinar cuánto tarda cada cosa.

Funciona con cualquier aplicación web
Los ejemplos usan Next.js, pero no depende de él.

Te ayuda a entender los fallos
Reportes, trazas y un modo visual para depurar.

La pirámide de pruebas

Unitarias

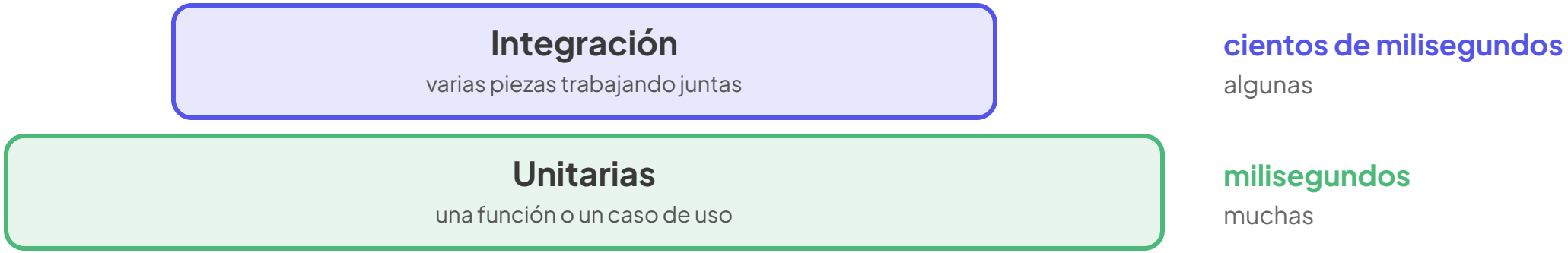
una función o un caso de uso

milisegundos

muchas

En la base, las pruebas **unitarias**: revisan una función o un caso de uso. Son rapidísimas y por eso hay muchas.

La pirámide de pruebas



En el medio, las de **integración**: varias piezas trabajando juntas, con las dependencias simuladas.

La pirámide de pruebas



Arriba, las **E2E**: lentas porque usan todo el sistema, así que son pocas y se reservan para lo importante.

Las E2E prueban el recorrido completo



Cuanto más arriba, más se parece la prueba a lo que vive un usuario, y más cuesta ejecutarla.

Universidad ICESI

Qué recorridos merecen una prueba E2E

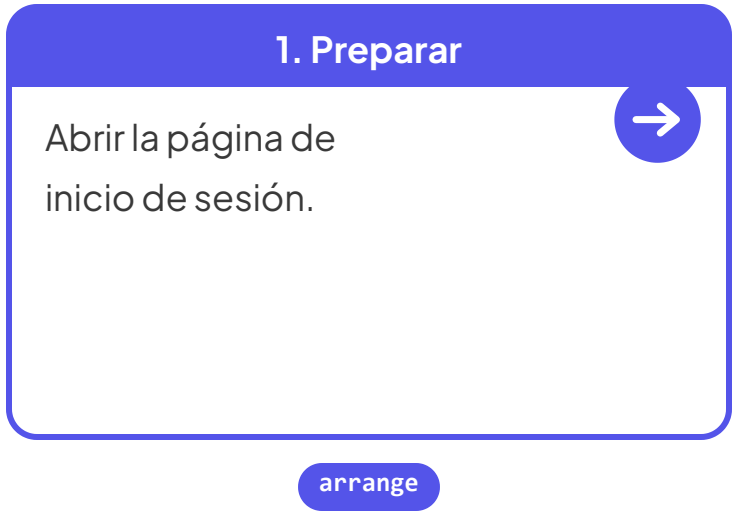
Iniciar sesión
Que el login funcione de punta a punta.

Que la sesión se guarde
La cookie existe y no la puede leer JavaScript.

Que una ruta protegida redirija
Entrar sin sesión debe llevar al login.

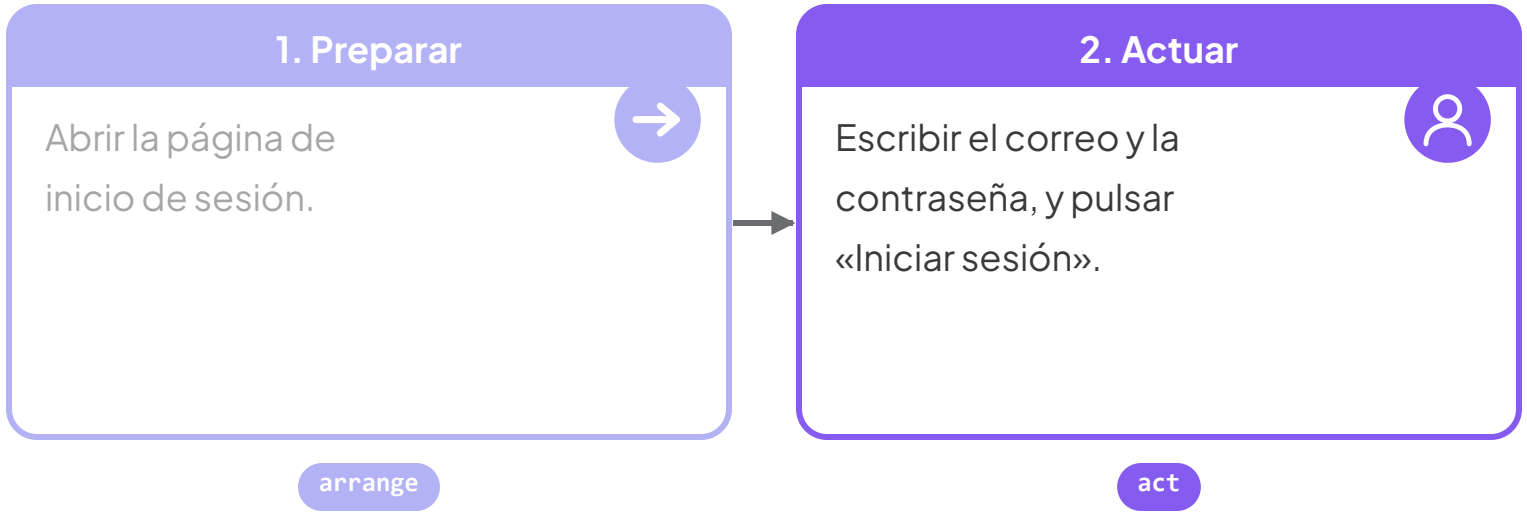
Crear algo y verlo en la lista
Un recorrido de negocio completo.

Una prueba tiene tres momentos



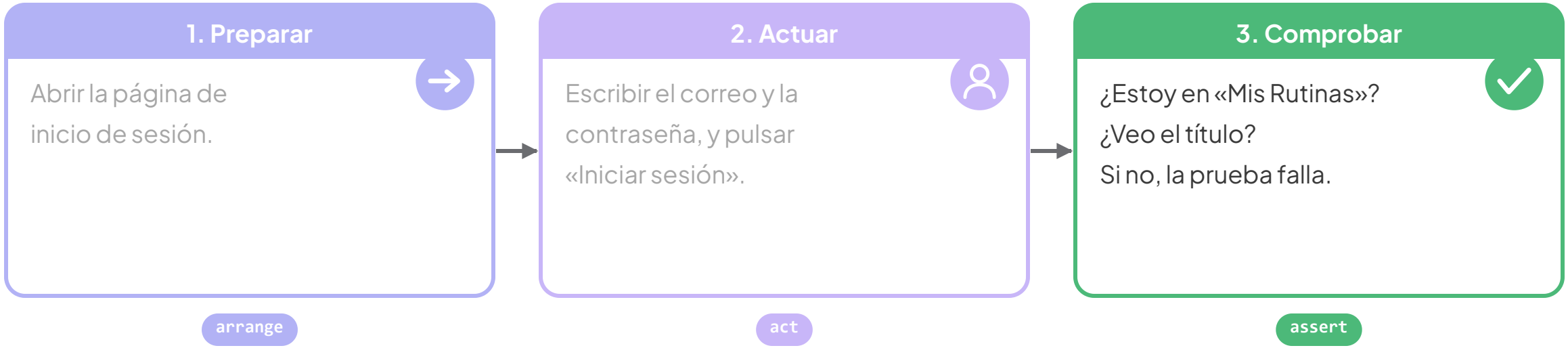
Primero se **prepara** el escenario: llegar al lugar desde donde empieza el recorrido.

Una prueba tiene tres momentos



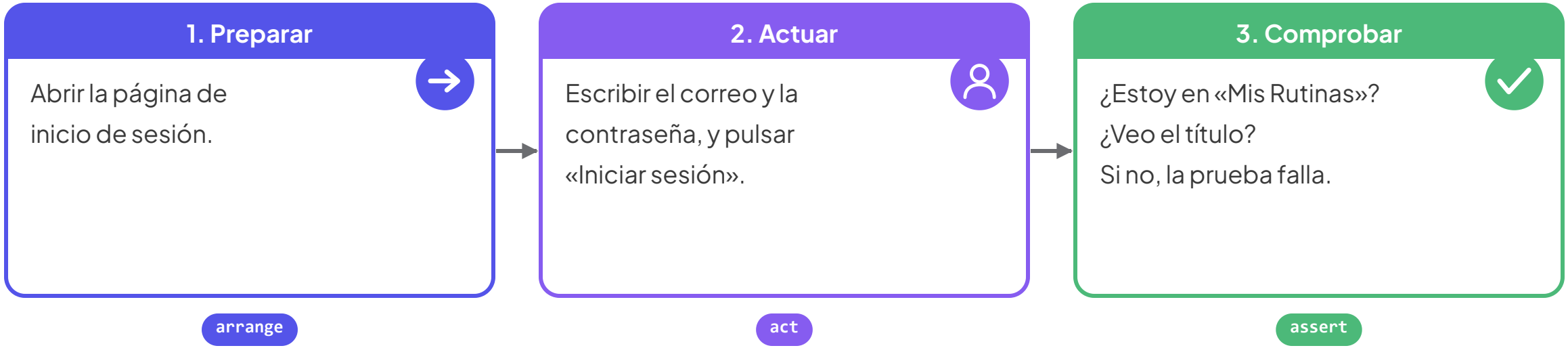
Después se **actúa** como lo haría una persona: escribir, pulsar, navegar.

Una prueba tiene tres momentos



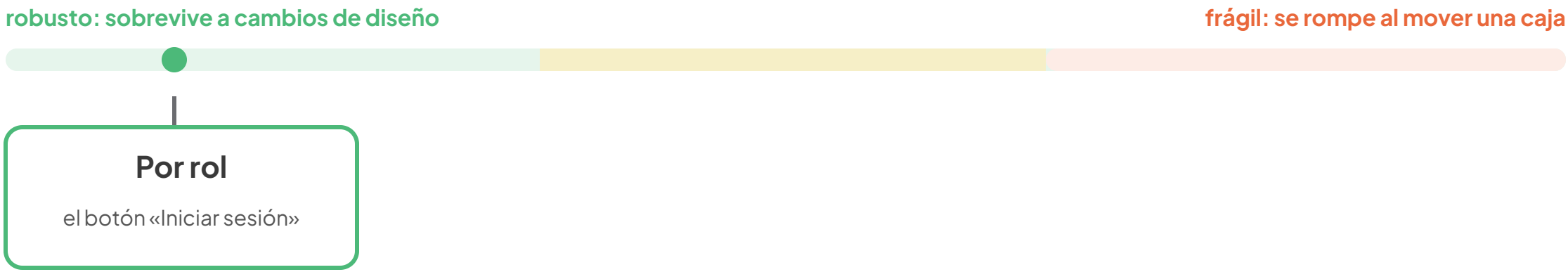
Por último se **comprueba** lo que se ve. Si lo esperado no aparece, la prueba falla.

Una prueba cuenta una historia



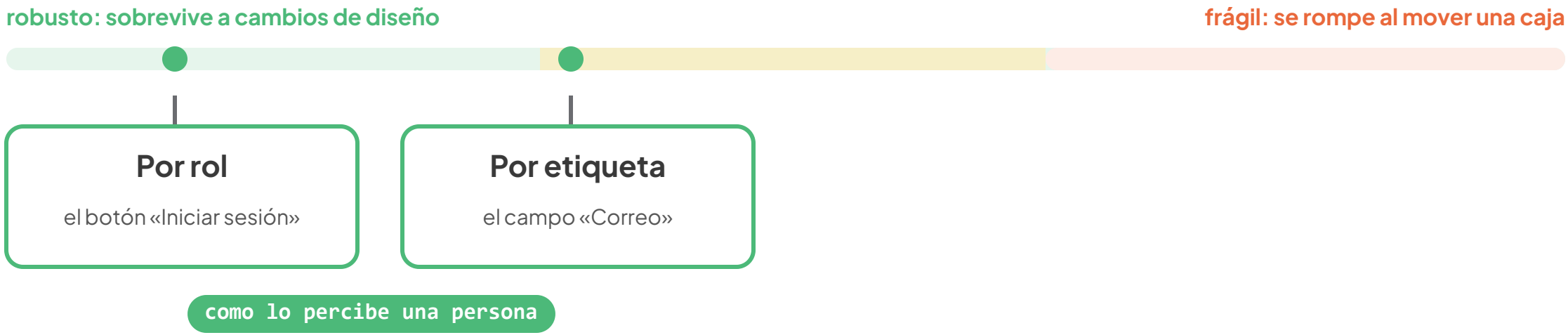
Preparar, actuar y comprobar: así se lee cualquier prueba, en cualquier herramienta.

Encontrar elementos como una persona



Para hacer clic o escribir, la prueba primero tiene que **encontrar** el elemento. Hay formas más robustas y más frágiles de hacerlo.

Mejor: por lo que la persona percibe



Por **rol** y por **etiqueta**: «el botón Iniciar sesión», «el campo Correo». Sobreviven a un rediseño.

Aceptable: por el texto visible



Por **texto**: sirve para contenido, aunque se rompe si cambian las palabras.

Frágil: por la estructura interna



Por **selector CSS**: depende de cómo está armada la página. Una caja nueva y la prueba falla aunque la app funcione.

Universidad ICESI

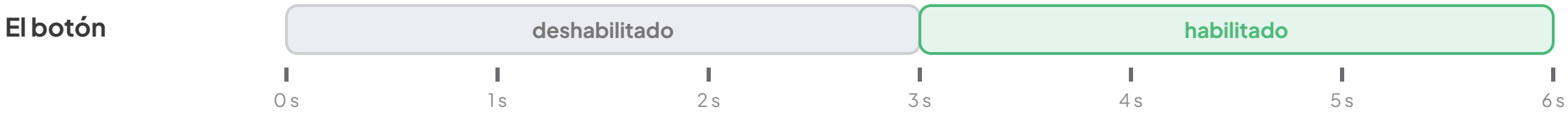


Una ventaja: apps accesibles

Localizar por rol y etiqueta funciona solo si la interfaz **las tiene**: un campo sin `label` es difícil de encontrar.

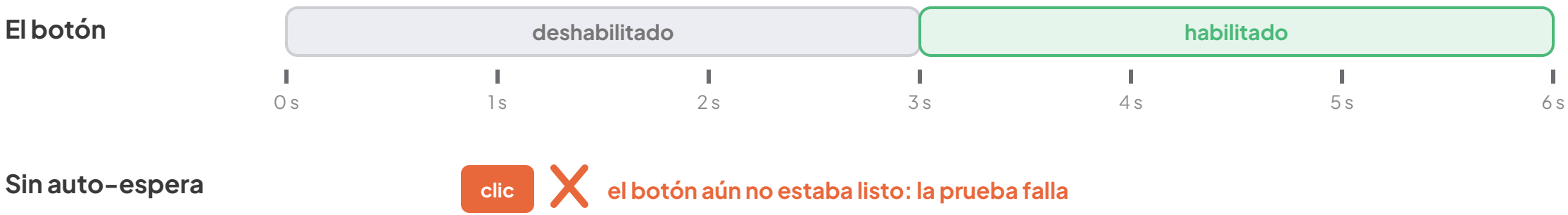
Así, escribir buenas pruebas te empuja a construir una interfaz **accesible** para todas las personas.

Las aplicaciones web son asíncronas



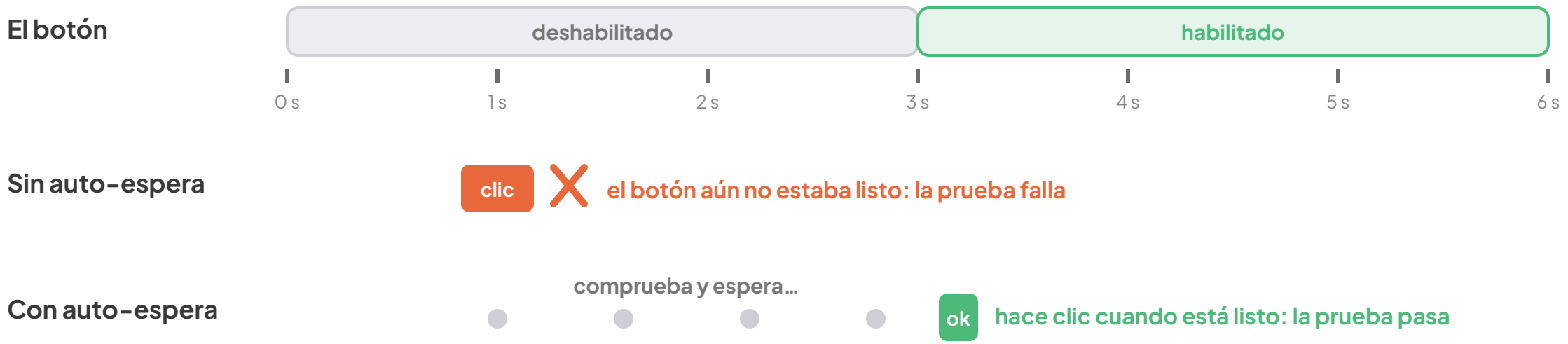
Un botón tarda en habilitarse, una página tarda en cargar. El tiempo no es predecible.

Sin auto-espera, la prueba falla sin motivo



Si la prueba actúa enseguida, llega antes de tiempo y falla aunque la aplicación esté bien: una prueba **inestable**.

Playwright espera por ti



Antes de actuar comprueba que el elemento esté listo, y las comprobaciones **reintentan** hasta que se cumplen o se acaba el tiempo.

Universidad ICESI

Cada prueba empieza con un navegador limpio

Cada prueba inicia sesión



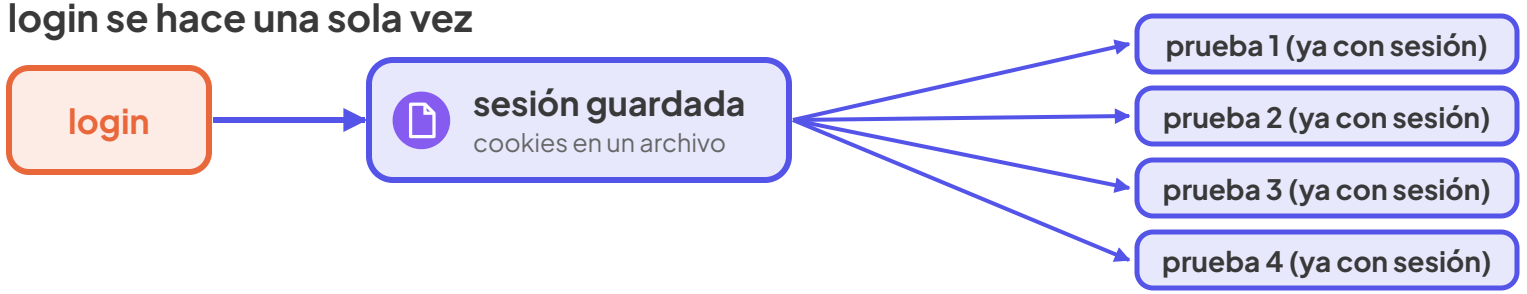
Sin cookies: para probar la parte protegida, cada prueba tendría que iniciar sesión primero. Es lento y repetitivo.

Se inicia sesión una sola vez

Cada prueba inicia sesión

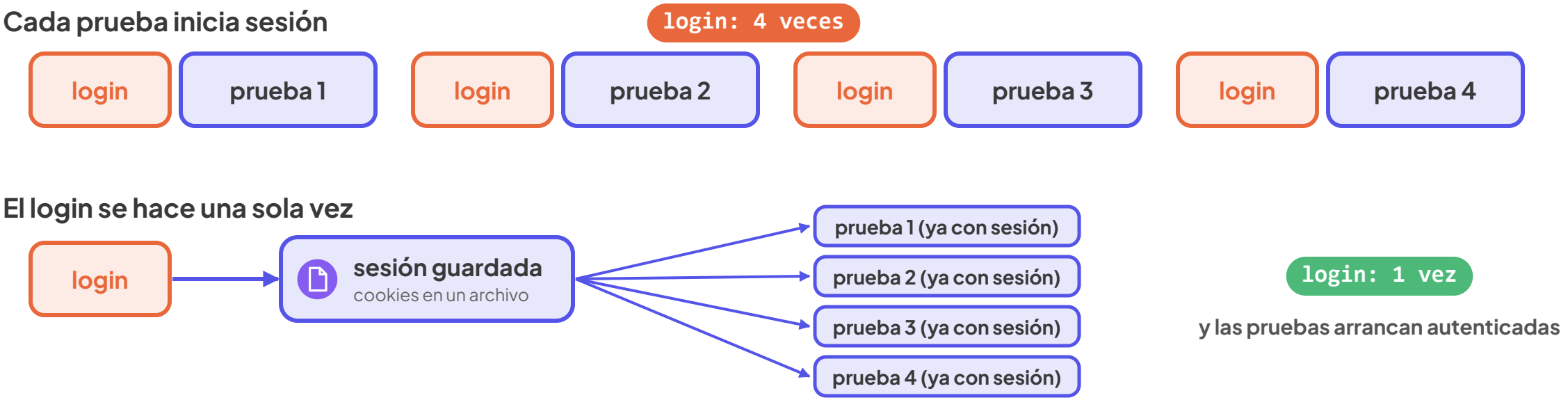


El login se hace una sola vez



Una preparación inicial hace el login y **guarda la sesión** en un archivo. Las pruebas la reutilizan y arrancan ya autenticadas.

Mucho menos trabajo, el mismo resultado



Las pruebas del login sí empiezan sin sesión; las demás parten autenticadas. Ese archivo tiene cookies reales: **nunca va al repositorio**.

Universidad ICESI

Cuando una prueba falla

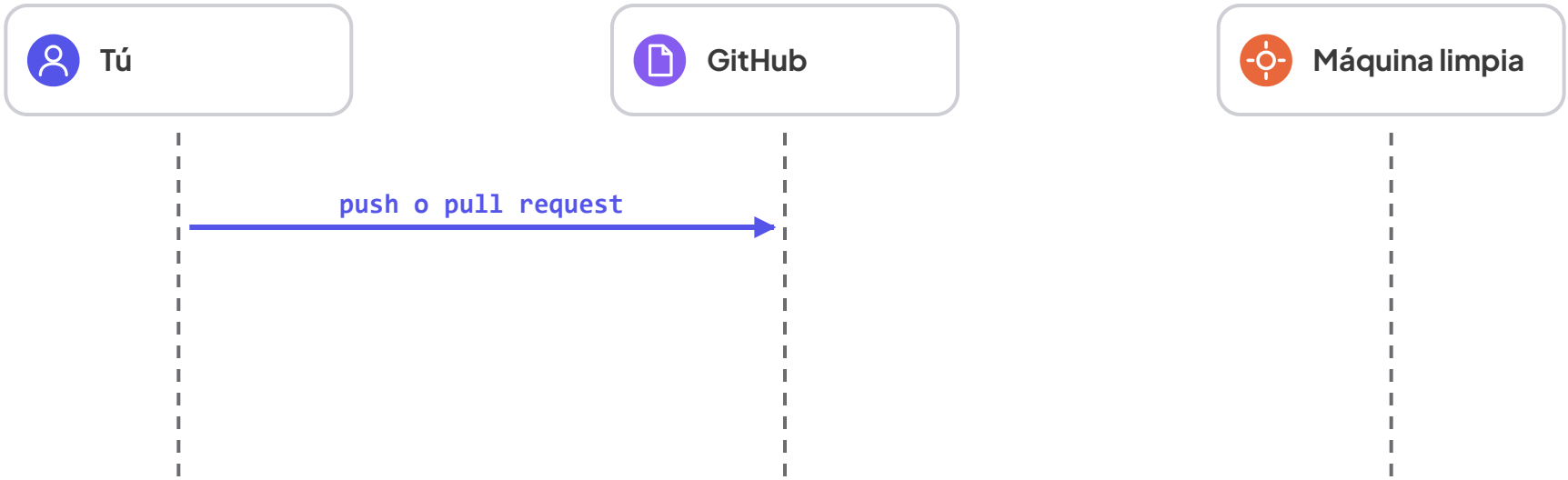
UI Mode
Recorres la prueba paso a paso, con una línea de tiempo y el estado de la página.

Trace Viewer
Repite una ejecución con lo que pasó en la página, la red y la consola.

Reporte HTML
Te cuenta qué pasó en la última ejecución.

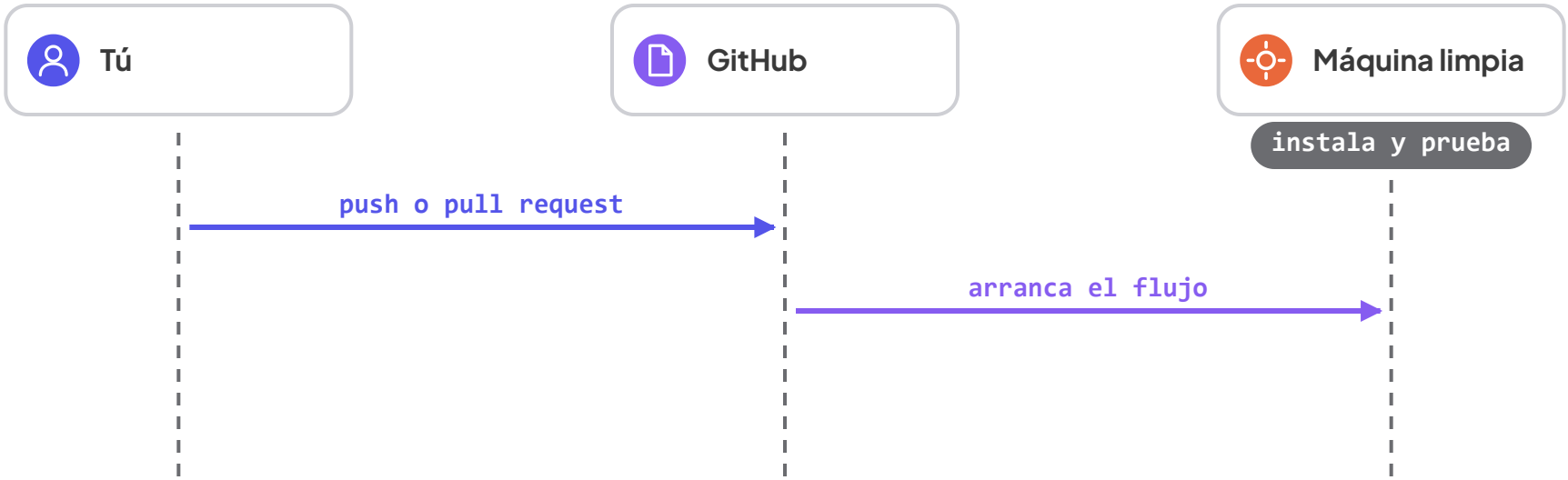
Codegen
Graba tus acciones en el navegador y te da el código de la prueba.

Integración continua (CI)



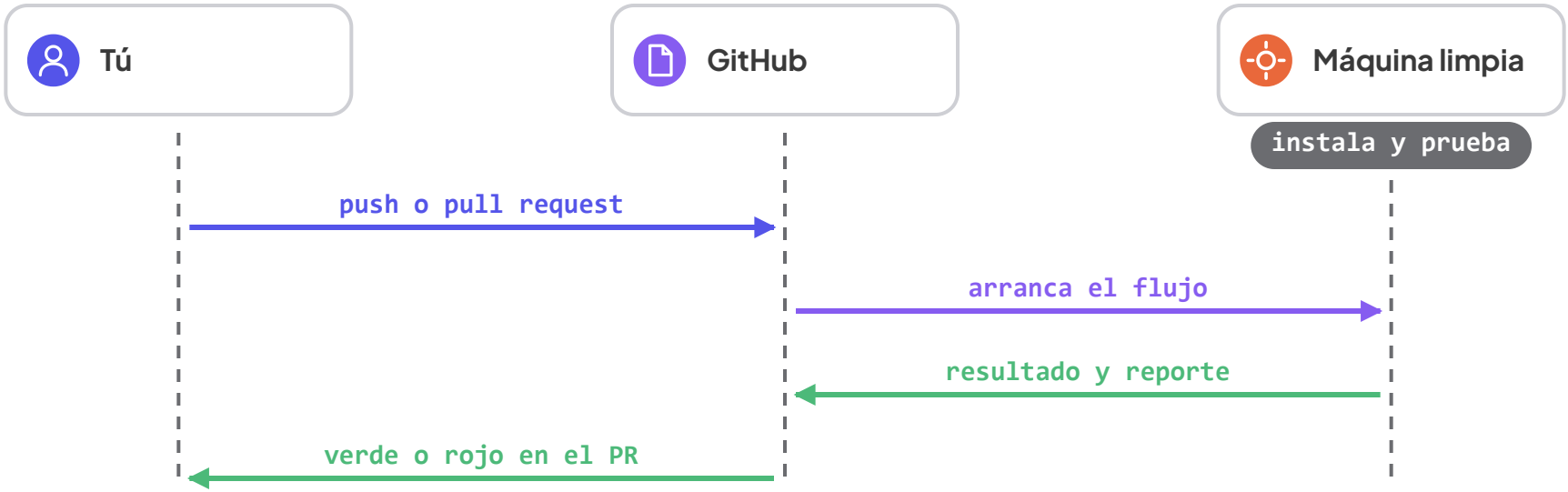
En cada cambio, GitHub se entera. El objetivo es probarlo **antes** de que llegue a la rama principal.

Una máquina limpia corre las pruebas



GitHub arranca una máquina sin nada tuyo instalado, justo como la vería otra persona: instala el proyecto, levanta la aplicación y ejecuta las pruebas E2E.

Y te cuenta el resultado



Un cambio que rompe el login se detecta **antes de llegar a main**, y el reporte queda guardado para revisarlo.

Universidad ICESI

2. Despliegue

Llevar la aplicación a internet

¿Qué es desplegar?

EN TU COMPUTADOR

Solo tú la ves.

- Vive en `localhost`.
- Funciona mientras tengas el servidor encendido.

DESPLEGADA

Cualquiera puede abrirla.

- Tiene una dirección pública con HTTPS.
- Corre en un servidor que no depende de tu equipo: es el entorno de **producción**.

CI y CD

CI · INTEGRACIÓN CONTINUA

Probar en cada cambio.

- Una máquina limpia instala el proyecto y ejecuta las pruebas.
- Es lo que hicimos con Playwright.

CD · ENTREGA CONTINUA

Publicar sin hacerlo a mano.

- Si todo pasó, el cambio se publica solo.
- Nadie sube archivos por su cuenta.

Universidad ICESI

Vercel

La plataforma de Next.js
Entiende un proyecto de Next.js sin configuración.

Construye y publica
Le da a tu aplicación una URL con HTTPS.

Una URL de preview por cada pull request
Revisas el cambio funcionando antes de aprobarlo.

Volver atrás es inmediato
Los despliegues no se sobrescriben: se conservan.

Dos formas de conectar GitHub con Vercel

INTEGRACIÓN DE GIT

Importas el repositorio y listo.

- Vercel construye y despliega solo.
- No ejecuta tus pruebas.

GITHUB ACTIONS + CLI

Tú escribes el flujo.

- Las pruebas corren primero.
- Si fallan, **no se despliega**.

El camino de un cambio



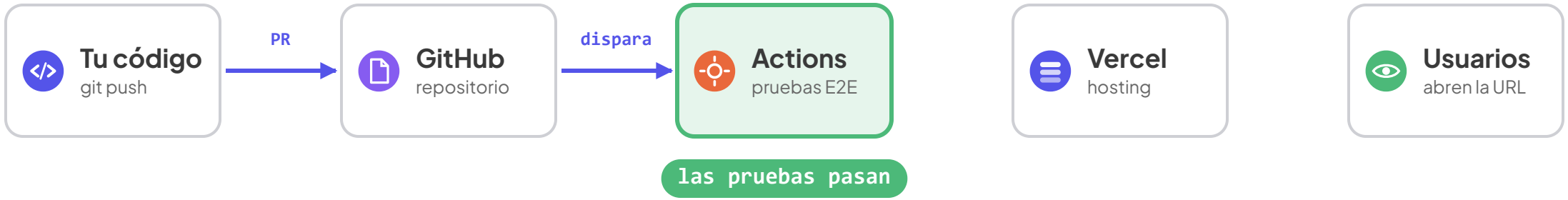
Abres un pull request: tu código llega a GitHub y GitHub lo registra.

El camino de un cambio



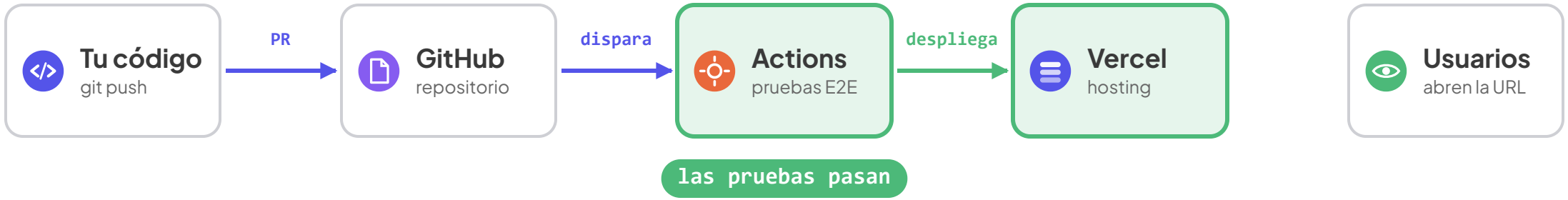
El archivo del flujo dice qué eventos arrancan el trabajo: GitHub lo ejecuta en una máquina limpia.

Primero, las pruebas



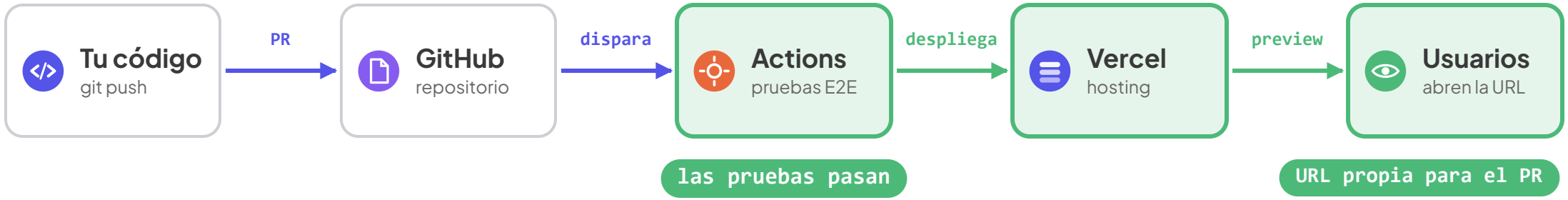
Playwright recorre el login y las rutas protegidas. Todo funciona como se esperaba.

Luego, el despliegue



Como las pruebas pasaron, el flujo entrega la aplicación a Vercel.

Cada pull request tiene su propia URL



Vercel publica una **versión de preview**: puedes revisar el cambio funcionando antes de aprobarlo.

Si las pruebas fallan, nada se despliega



Alguien hizo push a **main** y una prueba falla: algo del recorrido ya no funciona.

El despliegue se bloquea



El trabajo de despliegue depende del de pruebas: si falló, **no corre**.

Producción sigue igual



Los usuarios siguen viendo la versión anterior, que sí funcionaba. El error se arregla sin que nadie lo sufra.

Una aplicación, tres entornos

Development
Tu computador

`API_URL → localhost`

Preview
Ramas y pull requests

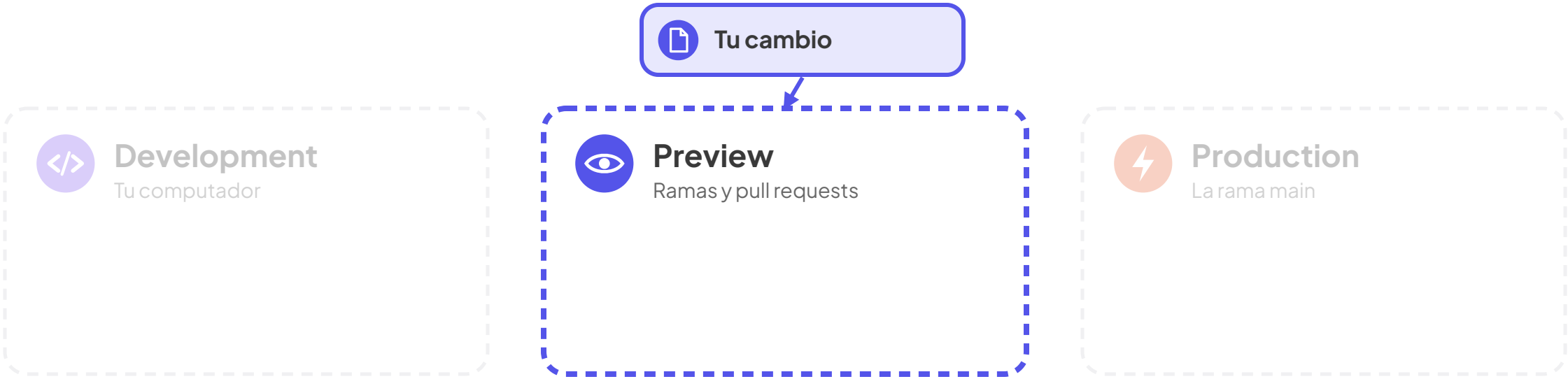
`API_URL → pruebas`

Production
La rama main

`API_URL → producción`

Vercel separa **Development** (tu computador), **Preview** (ramas y pull requests) y **Production** (la rama principal).

El entorno depende de dónde viene el cambio



Un pull request o una rama distinta de main crea un despliegue de **Preview**.

Cada entorno tiene sus propias variables



La misma aplicación se conecta a la API de pruebas en Preview y a la real en Production, sin cambiar el código.

Los cambios en main van a producción



Un cambio en la rama principal crea el despliegue de **Production**, con sus propios valores.

Universidad ICESI



Cambiar una variable

Un cambio en una variable **solo afecta a los despliegues nuevos**. El que ya está publicado conserva los valores con que se construyó.

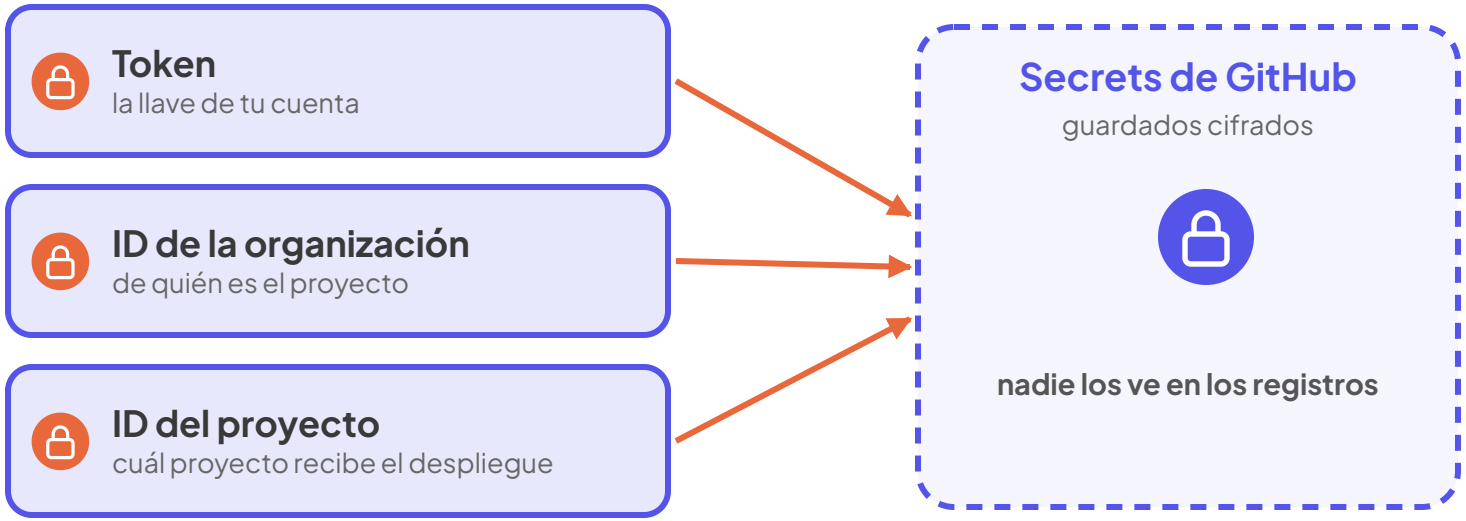
Para ver el cambio hay que **volver a desplegar**. Y `localhost` no sirve en producción: la API debe estar también desplegada.

El flujo necesita tres credenciales

- **Token**
  la llave de tu cuenta
- **ID de la organización**
  de quién es el proyecto
- **ID del proyecto**
  cuál proyecto recibe el despliegue

Para desplegar en tu nombre, el flujo necesita saber **quién eres** (el token), de qué **organización** y de qué **proyecto**.

Se guardan como secrets de GitHub



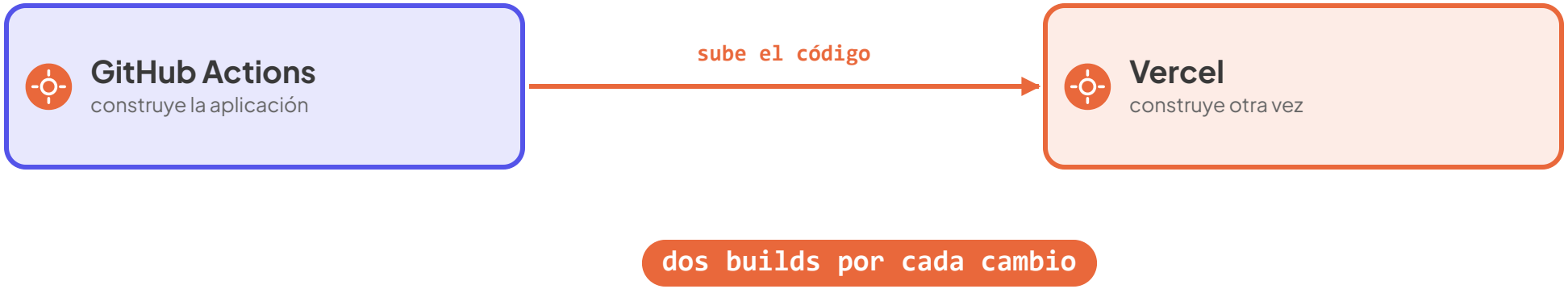
GitHub las guarda cifradas y no las muestra en los registros. Por eso no se escriben nunca en el código.

El flujo las usa sin revelarlas



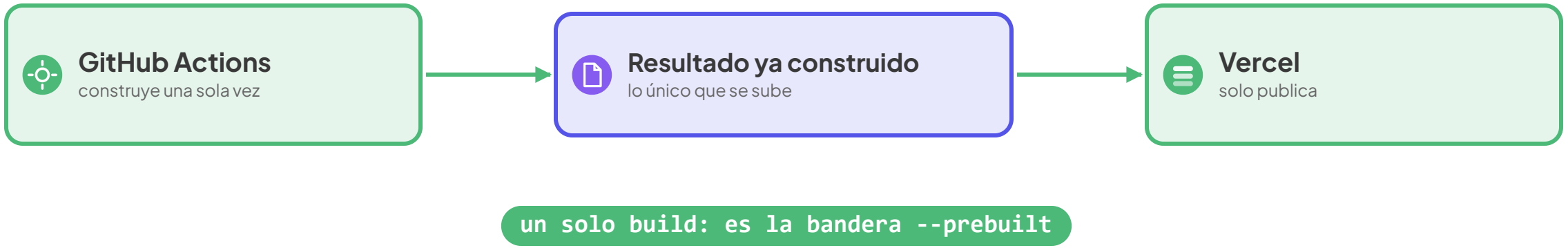
Quien tenga el token puede desplegar como tú: si alguna vez queda expuesto, hay que **revocarlo** y crear otro.

Dos formas de construir



Si el flujo construye y Vercel vuelve a construir lo mismo, hay **dos builds por cada cambio**: más lento y más caro.

Construir una sola vez



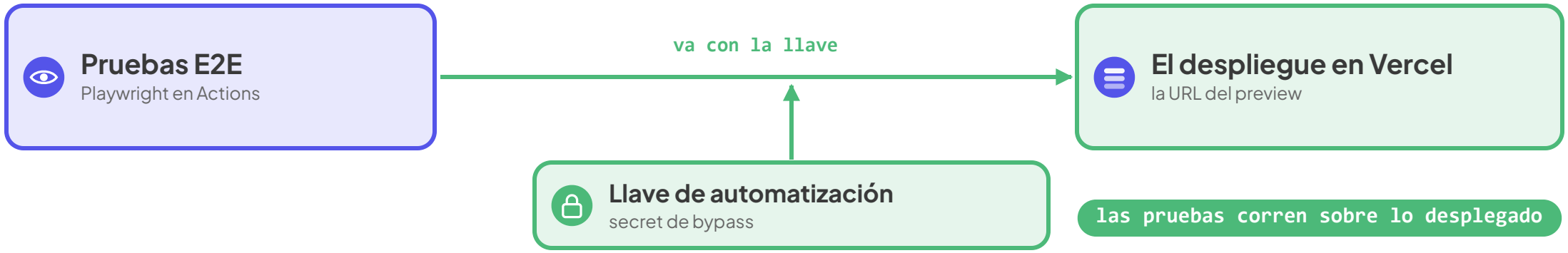
Con la bandera `--prebuilt`, el flujo construye y Vercel **solo publica** el resultado ya compilado.

Probar lo que de verdad llega a los usuarios



Las pruebas del flujo usan una copia de la app. Lo que ve el usuario es el despliegue, pero los previews suelen estar protegidos.

Una llave de automatización



Con una **llave de automatización**, Playwright entra al preview y ejecuta las mismas pruebas sobre lo desplegado.

Si producción sale mal



La versión 3 llegó a producción con un error. Los despliegues anteriores siguen ahí, ya construidos.

Volver atrás es inmediato



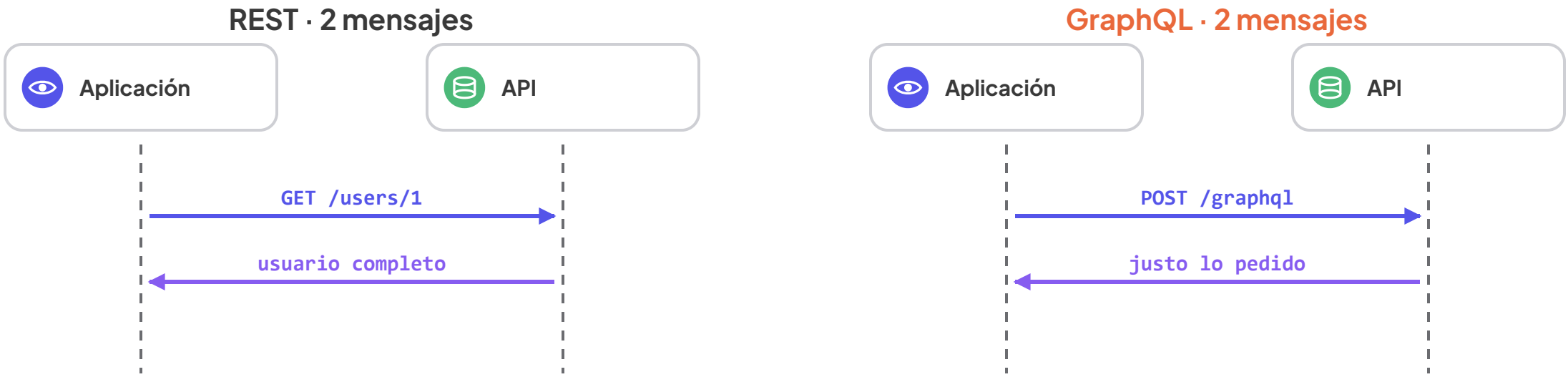
Producción apunta de nuevo a la versión 2. No se reconstruye nada: **solo cambia un puntero**.

Universidad ICESI

3. GraphQL

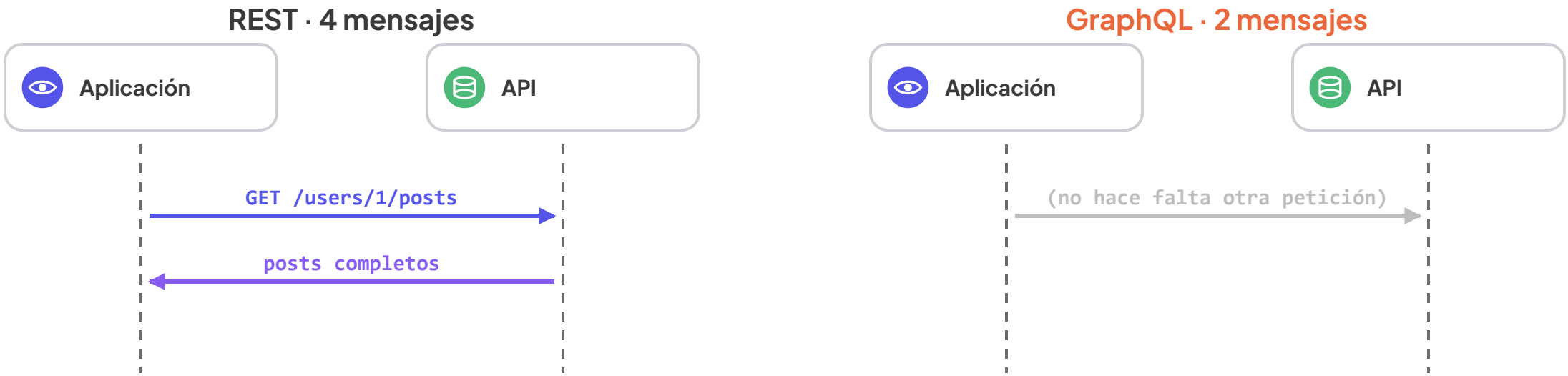
Pedir exactamente lo que necesitas

La misma pantalla, dos formas de pedirla



La pantalla necesita el **nombre** de un usuario y los **títulos de sus posts**. REST empieza por el recurso del usuario; GraphQL describe todo lo que necesita.

REST sigue pidiendo



Los posts son otro recurso: REST necesita **otra petición**. GraphQL ya trajo los títulos junto con el usuario.

Resultado

REST

2 peticiones

y campos de sobra

GraphQL

1 petición

y exactamente lo pedido

GraphQL mueve la decisión de **qué datos llegan** del servidor al cliente.

Dos problemas clásicos de REST

OVER-FETCHING

Traer de más.

- `GET /users/1` devuelve todos los campos del usuario.
- La pantalla solo usa el nombre.

UNDER-FETCHING

Traer de menos.

- Un endpoint no alcanza.
- Hay que encadenar peticiones: el usuario, sus posts, los comentarios…

Universidad ICESI

Qué es GraphQL

Un lenguaje de consulta para APIs
El cliente describe los datos que necesita.

Un sistema de tipos
El servidor publica un schema: qué datos existen y cómo se relacionan.

Normalmente, un solo endpoint
Por ejemplo /graphql: la consulta viaja en el cuerpo de la petición.

No es una base de datos
Es una capa entre tus clientes y tus datos, vengan de donde vengan.

Universidad ICESI



Nació en Facebook

Lo creó Facebook en **2012** para sus aplicaciones móviles, donde cada petición de más cuesta batería y datos.

Se liberó como código abierto en **2015** y hoy lo mantiene la GraphQL Foundation.

El schema: el contrato

type User

```
id: ID!
name: String!
email: String!
posts: [Post!]!
```

El servidor describe sus datos con **tipos**. Un `User` tiene un identificador, un nombre, un correo y una lista de posts.

Los tipos se relacionan

type User

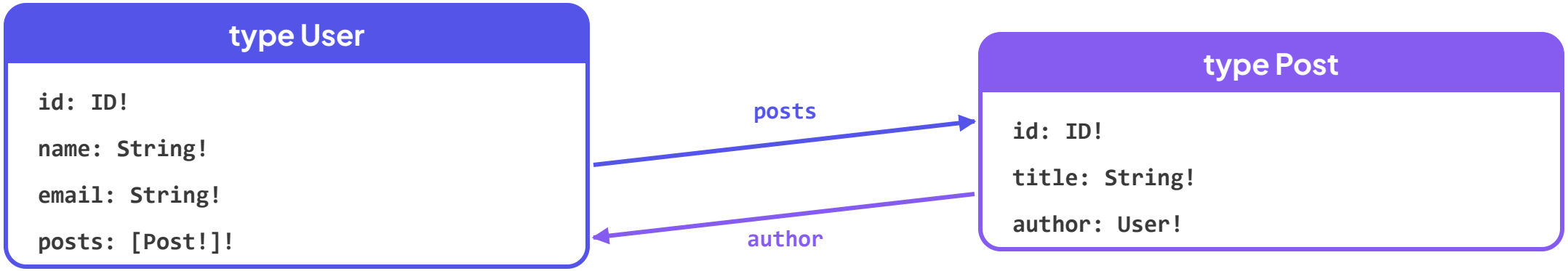
```
id: ID!
name: String!
email: String!
posts: [Post!]!
```

type Post

```
id: ID!
title: String!
author: User!
```

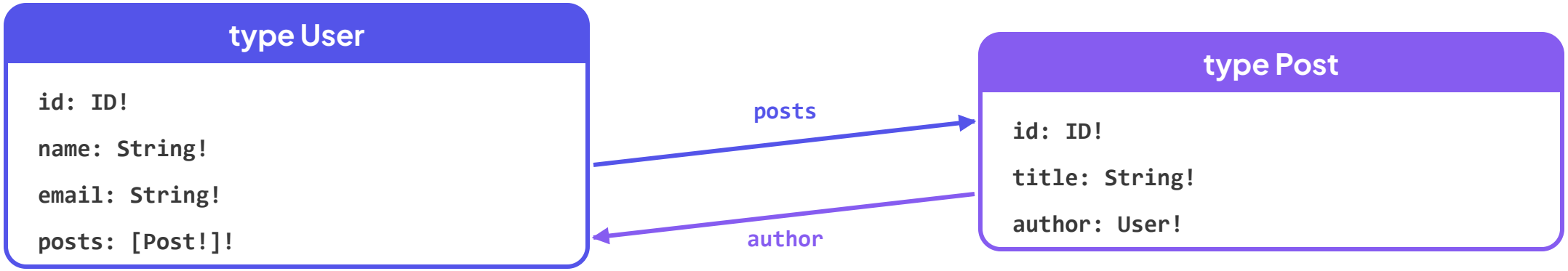
Un `Post` tiene un título y un autor. Cada tipo se define una sola vez.

Un grafo de datos



Las relaciones forman un **grafo**: del usuario a sus posts y de cada post a su autor. Así se puede recorrer en una sola consulta.

Obligatorio o no



`! significa obligatorio: nunca es null`

Por defecto todo campo puede ser `null`. El signo `!` lo vuelve **obligatorio**, y los corchetes indican una lista.

Las puertas de entrada

QUERY

Leer datos.

Es lo que hará la mayoría de tus operaciones.

MUTATION

Escribir: crear, actualizar o borrar.

Se ejecutan una tras otra, en orden.

SUBSCRIPTION

Recibir datos **en tiempo real** cuando algo cambia.

Por ejemplo, un chat.

Pides con una forma, recibes esa forma

Consulta (lo que pides)

```
{
  user {
    name
  }
}
```

servidor GraphQL

Respuesta (la misma forma)

```
{ "user": {
    "name": "Ana"
} }
```

Una consulta parece un JSON sin valores: escribes la forma que quieres y la respuesta tiene **exactamente esa forma**.

Universidad ICESI

Se recorre el grafo en una sola petición

Consulta (lo que pides)

```
{
  user {
    name
    posts {
      title
    }
  }
}
```

servidor GraphQL →

Respuesta (la misma forma)

```
{ "user": {
    "name": "Ana",
    "posts": [
      { "title": "Hola" },
      { "title": "Otro" }
    ]
} }
```

Pidiendo también los títulos de los posts, la misma petición trae al usuario **y** sus posts.

El schema valida lo que pides

Consulta (lo que pides)

```
{
  user {
    telefono
  }
}
```

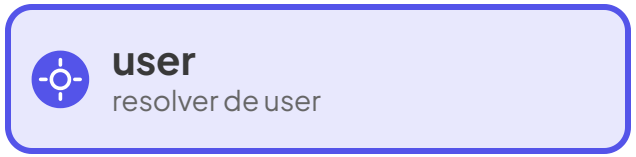
el schema la rechaza

Respuesta (un error)

```
{ "errors": [
    "No existe el campo
     telefono en User"
] }
```

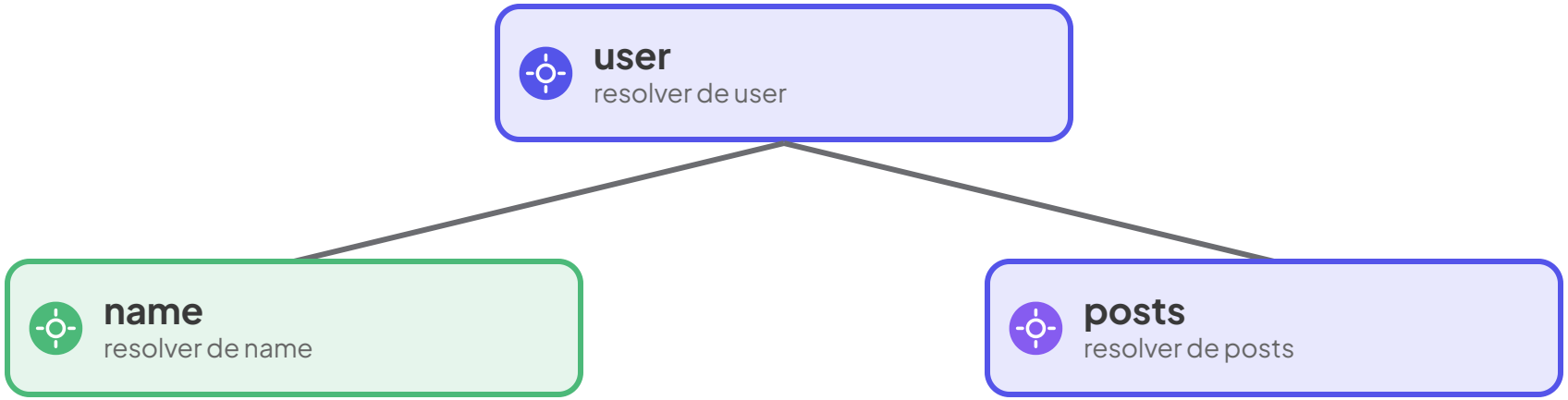
Si pides un campo que no existe, la consulta se **rechaza antes de ejecutar nada**, con un mensaje que dice exactamente qué falló.

Cómo se ejecuta: resolvers



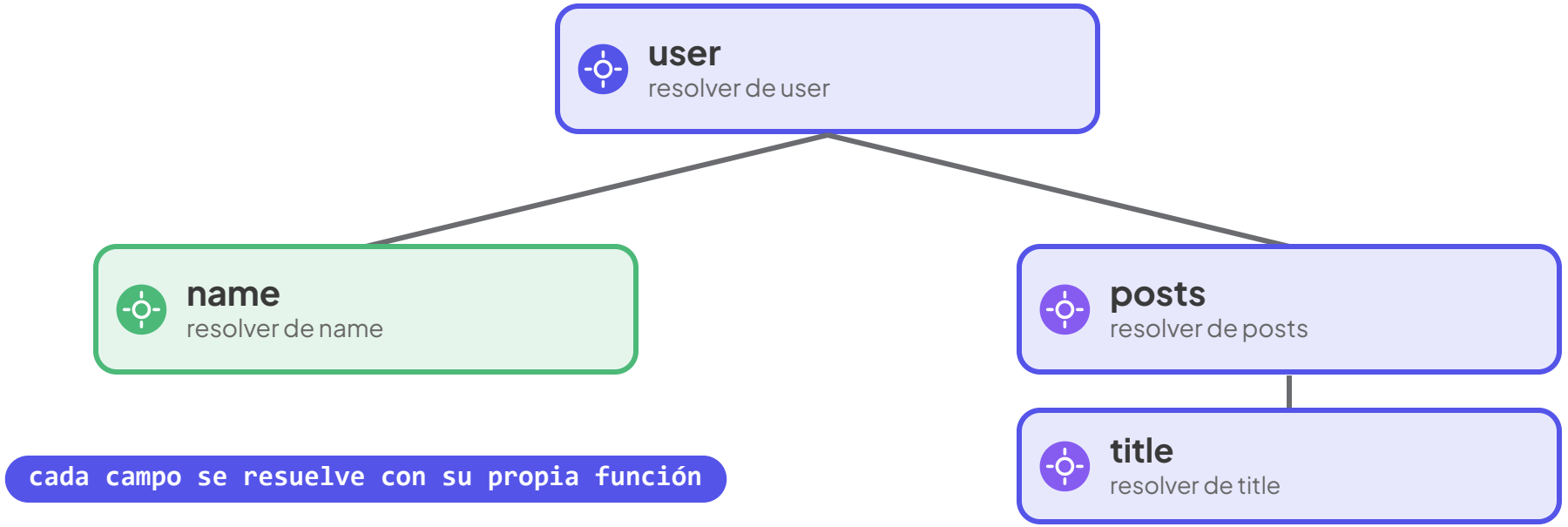
Después de validar, el servidor ejecuta la consulta. Cada campo tiene una función propia, un **resolver**.

Cada campo, su función



El resolver de `user` trae al usuario; los de `name` y `posts` completan lo que se pidió.

Solo se calcula lo pedido



Los campos que no pediste **no se resuelven**. El resultado se arma con la misma forma que la consulta.

Universidad ICESI



Introspección y sandbox

Un servidor GraphQL **se describe a sí mismo**: puedes consultarle su propio schema.

Por eso herramientas como **Apollo Sandbox** o GraphiQL autocompletan, validan y muestran la documentación sin que nadie la escriba aparte.

Universidad ICESI

Ventajas de GraphQL

Pides lo justo
Sin campos de sobra y sin peticiones encadenadas.

El contrato se documenta solo
Los errores se detectan al validar contra el schema.

Evolución sin versiones
Agregas campos sin romper a nadie y marcas los viejos como obsoletos.

Los equipos avanzan sin esperar
El frontend no necesita endpoints nuevos para cada pantalla.

Desventajas de GraphQL

Una consulta puede ser muy costosa
Si no se limita, alguien puede pedir listas dentro de listas sin tope.

El servidor es más complejo
Hay que escribir y mantener el schema y los resolvers.

La caché HTTP rinde menos
Todas las peticiones van al mismo endpoint.

El problema N+1
Un resolver por elemento puede multiplicar las consultas a la base de datos.

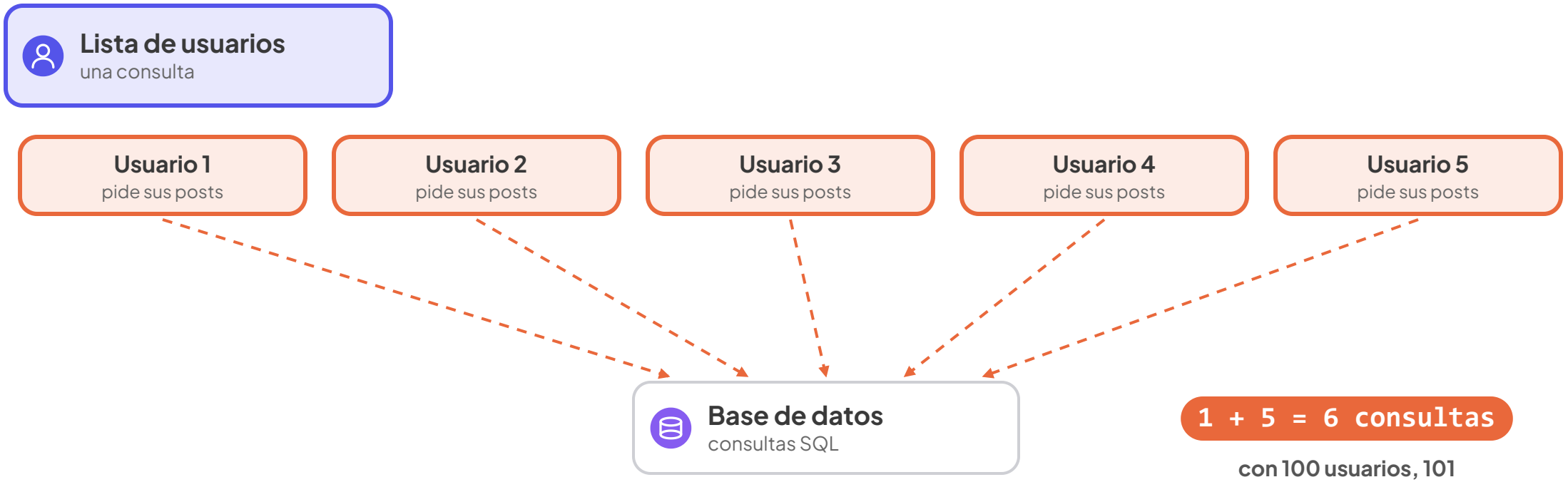
El problema N+1



Pides una lista de usuarios y, para cada uno, sus posts. Primero se trae la lista: **una consulta**.

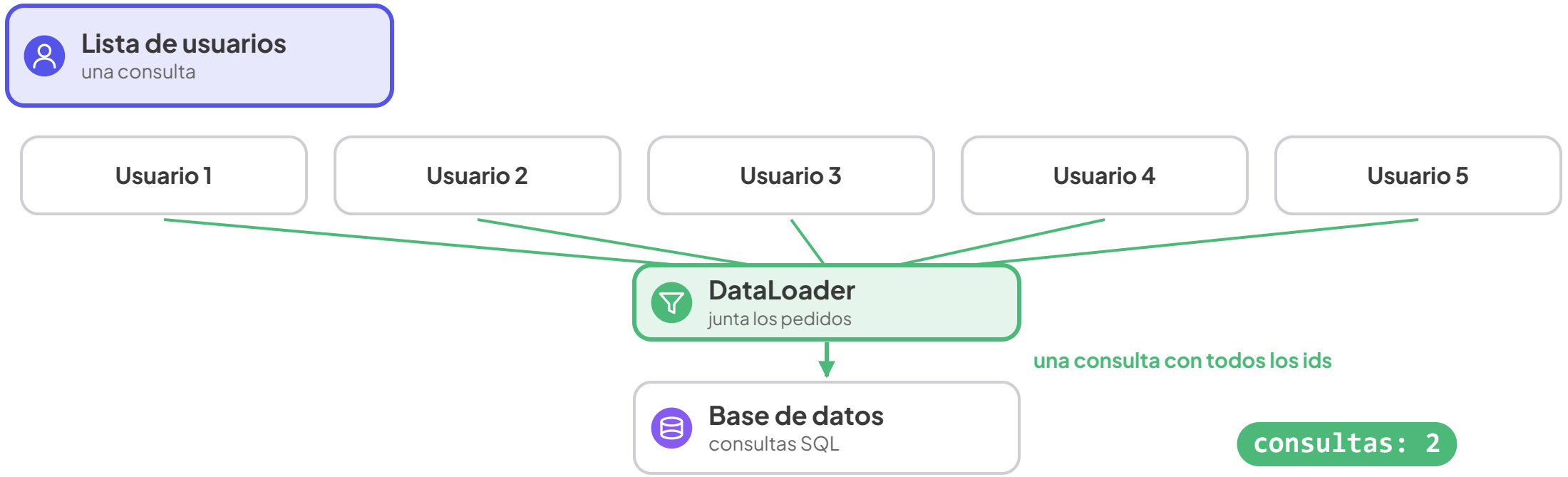
Universidad ICESI

Una consulta por cada usuario



El resolver de `posts` se ejecuta una vez por usuario y cada vez pregunta a la base de datos: 1 para la lista y N para sus relaciones.

Se resuelve agrupando



Un **DataLoader** junta los pedidos de la misma ronda y hace una sola consulta con todos los ids: siempre 2 consultas, sin importar N.

¿GraphQL o REST?

CONVIENE GRAPHQL CUANDO…

- Hay **varios clientes** (web, móvil) con necesidades de datos distintas.
- Las pantallas combinan datos de muchos recursos relacionados.
- El frontend cambia rápido.

CONVIENE REST CUANDO…

- La API es simple y los recursos son pocos y estables.
- Aprovechas la **caché HTTP** y los CDN.
- Quieres algo sencillo de operar y monitorear.